



Software Engineering Department
Braude College

Capstone Project Phase B – 61998

Github link: https://github.com/idan-aizenberg/Final_project.git

WeatherSight- Long term weather forecast marketing platform

25-2-D-24

Students:

Idan Aizenberg

Aviv Aytok

Lecturer:

Zekharya Frenkel

Table of Contents

Review of the Problem and Background	
1.1 Abstract	3
1.2 Introduction	4
1.3 Literature Review Summary	5
Project Description and Achievements	
2.1 General Description	6
2.2 Solution Description and System Architecture	7
2.3 Development Process	8
2.4 Tools and Technologies	9
2.5 Engineering Challenges and Solutions	10
2.6 Results and Conclusions	11
2.7 Lessons Learned	11
2.8 Project Metrics and Goal Validation	12
2.9 Summary and Future Work	14
Appendix A: User Guide (Operating Instructions)	
A.1 System Overview	15
A.2 Getting Started	15
A.3 Core System Operations	16
A.4 Managing User Data	17
A.5 Data Visualization	18
A.6 Summary	18
Appendix B: Maintenance Guide (Developer Instructions)	
B.1 Purpose	19
B.2 System Environment Requirements	19
B.3 Installation and Initial Setup	20
B.4 Running the System	21
B.5 System Maintenance	21
B.6 Monitoring and Troubleshooting	22
B.7 Scalability Considerations	22
B.8 Security Considerations	23

1. Review of the Problem and Background

1.1 Abstract

WeatherSight is a web-based platform for accessing and analyzing long-term weather forecast data, designed to address the growing need for reliable climate information in weather-sensitive industries on a global scale. While recent advances in climate modeling have significantly improved extended-range forecasting capabilities, the practical use of such data remains limited. Forecast information is often fragmented across multiple forecasting providers, stored in heterogeneous formats, and presented in ways that are difficult for non-expert users to interpret and apply.

The core challenges addressed by this project are threefold. First, data fragmentation, where relevant climate datasets are distributed across isolated and often proprietary sources operated by different forecasting organizations. Second, limited accessibility, as many forecasting providers focus primarily on model accuracy and scientific output, but lack the infrastructure required to deliver user-friendly, scalable, and globally accessible platforms. Third, an actionability gap, in which probabilistic long-term forecasts are not translated into insights that support concrete operational decisions.

WeatherSight addresses these challenges by aggregating large-scale forecast data into a unified, provider-agnostic system that supports global coverage. The platform offers location-based search, interactive visualization, and structured access control, while remaining independent of any specific forecasting company. This design allows WeatherSight to function as a generic interface layer that can be adopted by different forecasting providers or used as a marketplace-style platform. The system aims to make long-term weather information more accessible, interpretable, and usable for decision-making in domains such as agriculture, energy planning, and logistics worldwide.

1.2 Introduction

In recent years, increasing climate variability has amplified the importance of long-term weather forecasting for businesses and organizations operating across different regions of the world. Decisions related to crop planning, energy production, supply chain management, and infrastructure investment often depend on expected weather conditions months in advance. Despite this need, most commercially available weather applications continue to focus on short-term forecasts and localized use cases, offering limited support for long-range and global planning.

At the same time, specialized climate and meteorological organizations continue to develop sophisticated forecasting models capable of producing extended-range predictions at a global scale. These models are often produced by different providers, each using distinct data formats, resolutions, and access mechanisms. As a result, organizations seeking to leverage long-term forecasts are required to interact with multiple systems or rely on raw data, which introduces technical and operational barriers.

WeatherSight was conceived as a response to this gap between high-quality, globally available forecast data and practical data consumption. The system functions as a centralized, provider-independent platform through which users can access long-term forecast data in a consistent and structured manner, regardless of the underlying forecasting source. Rather than requiring users to interact directly with raw datasets or provider-specific tools, WeatherSight abstracts the underlying complexity and presents the data through intuitive search mechanisms and visual analytics.

By separating the data consumption layer from the forecasting provider layer, WeatherSight supports informed decision-making while reducing the technical barrier associated with advanced climate data. This approach allows the platform to scale globally and remain adaptable to different forecasting companies and data sources.

1.3 Literature Review Summary

Academic and applied research consistently highlights the value of long-term weather and climate forecasts across multiple sectors. Unlike short-term forecasting, which typically focuses on deterministic outcomes, long-term forecasting relies on probabilistic models and trend analysis. These forecasts are particularly relevant in contexts where strategic planning outweighs immediate operational responses.

Studies in the agricultural domain emphasize the role of seasonal and long-range forecasts in optimizing planting schedules, irrigation planning, and yield estimation. In the energy sector, long-term forecasts support demand prediction, renewable energy integration, and grid stability planning. Similarly, research in logistics, insurance, and infrastructure management demonstrates the importance of climate-informed risk assessment and resilience planning.

Despite their potential value, existing solutions often fail to bridge the gap between complex scientific outputs and practical user needs. Many platforms prioritize model accuracy over usability, offering limited tools for exploration, comparison, or visualization. As a result, decision-makers without specialized meteorological expertise struggle to extract actionable insights.

WeatherSight builds upon these findings by focusing not on improving forecast accuracy itself, but on improving access, usability, and interpretability. The platform translates large-scale, probabilistic weather data into an interface designed to support exploration, comparison, and long-term planning, directly addressing limitations identified in existing literature.

In addition, several studies emphasize the growing need for intermediary platforms that bridge the gap between forecast producers and forecast consumers. Research highlights that organizations often struggle not due to lack of data accuracy, but due to lack of standardized access, interoperability, and decision-oriented presentation. These findings support the need for provider-agnostic platforms that can integrate data from multiple forecasting sources and present it through a unified interface. WeatherSight directly aligns with this approach by focusing on usability, accessibility, and global applicability rather than on the development of proprietary forecasting models.

2. Project Description and Achievements

2.1 General Description

WeatherSight is a web-based platform designed to provide long-term weather forecasts in an accessible and visual manner. Unlike conventional weather applications that focus on short-term predictions, WeatherSight enables users to explore forecast data for periods of up to 365 days ahead. The system is intended primarily for users operating in weather-sensitive domains such as agriculture, logistics, energy planning, and infrastructure management, where decisions often need to be made months in advance.

The platform aggregates large-scale meteorological datasets and exposes them through a searchable interface that supports global coverage. Users can query any geographic location worldwide and receive forecast information based on the nearest available weather grid point.

Key system capabilities include:

- Global grid-based search: allowing users to query weather data for any coordinate on Earth using a predefined grid of approximately 13,000 points.
- Interactive visualizations: including charts and maps that present long-term trends in temperature and precipitation.
- Subscription-based access control: implemented through a tiered model (Free, Pro, Enterprise), which limits forecast range and data granularity according to user privileges.
- Personalized user dashboard: where users can manage saved searches, alerts, and account-related settings.

Together, these features aim to transform complex and large-scale weather data into a practical decision-support tool.

WeatherSight is a web-based platform designed to provide long-term weather forecasts in an accessible and visual manner, with global applicability. Unlike conventional weather applications that focus on short-term predictions or region-specific services, WeatherSight enables users to explore forecast data for periods of up to 365 days ahead for any geographic location worldwide. The system is intended primarily for users operating in weather-sensitive domains such as agriculture, logistics, energy planning, and infrastructure management, where decisions often need to be made months in advance and across multiple regions.

The platform is designed to be forecast-provider agnostic. It does not depend on a single forecasting company or proprietary model, but instead operates as a generic interface layer capable of integrating forecast data generated by different providers. This design allows WeatherSight to function either as an independent analytics platform or as a distribution and visualization layer for multiple forecasting organizations.

2.2 Solution Description and System Architecture

WeatherSight is implemented as a modern Single Page Application (SPA) built on the Next.js framework. This architecture allows tight integration between the frontend user interface and backend services while maintaining a clear separation of concerns.

At a high level, the system consists of the following components:

- **Frontend:**
A React-based user interface developed using Next.js and styled with Tailwind CSS and shadcn/ui components. The frontend is responsible for user interaction, data visualization, and dashboard functionality. It communicates with backend services through RESTful API endpoints.
- **Backend:**
Backend logic is implemented using Next.js Serverless API Routes. These routes handle request validation, authentication checks, query orchestration, and communication with the database. This approach simplifies deployment while keeping backend logic closely aligned with the frontend.
- **Database:**
Supabase (PostgreSQL) serves as the primary data store. The database includes several core tables:
 - `weather_grid`, which stores approximately 13,000 global coordinate points.
 - `weather_forecasts`, which stores daily forecast data linked to grid points, resulting in millions of rows.
 - `users` and `subscriptions`, which manage authentication and access control.

- **Provider Independence and Global Design Considerations:**
The system architecture intentionally separates data ingestion and storage from data consumption and visualization. Forecast data is stored in a standardized internal schema, allowing WeatherSight to support multiple forecasting sources without coupling the platform to a specific provider. This abstraction enables the system to operate at a global scale and simplifies the integration of additional datasets or forecasting companies in the future, without requiring architectural changes to the core platform.
- **Spatial Search Engine:**
To efficiently map user queries to the nearest weather grid point, the system implements a KD-tree (k-dimensional tree) data structure within the application layer. This spatial index enables fast nearest-neighbor searches with logarithmic time complexity, significantly improving performance compared to naive approaches.

An architecture diagram illustrating these components and their interactions is provided in Figure X.

2.3 Development Process

The development of WeatherSight followed an iterative and incremental process inspired by Agile principles. Instead of attempting to build the entire system in a single development cycle, the project was divided into logical stages, each producing a functional and testable subsystem.

The main development phases included:

1. **Research and Planning (Phase A)** – requirement definition, background research, and high-level architectural design.
2. **Data Ingestion and Schema Design** – development of Python and Node.js scripts to parse raw meteorological data files and populate the PostgreSQL database.
3. **Core System Implementation** – creation of the Next.js application structure, routing, and initial authentication mechanisms.
4. **Feature Development** – implementation of search logic, interactive maps (Leaflet), and data visualizations (Recharts).

5. **Optimization and Refinement** – performance tuning of database queries and integration of the KD-tree for spatial indexing.
6. **Integration and Testing** – manual testing of workflows based on defined user personas and iterative refinement of usability and performance.

This staged approach allowed early validation of architectural decisions and reduced the risk of late-stage redesign.

2.4 Tools and Technologies

The project was implemented using the following technologies:

Frontend

- Next.js 14 (React 18)
- TypeScript
- Tailwind CSS and shadcn/ui
- Recharts for data visualization
- React Leaflet for geographic visualization

Backend and Data

- Supabase (PostgreSQL)
- Next.js API Routes
- KD-tree spatial indexing algorithm

2.5 Engineering Challenges and Solutions

Challenge 1: Database Size and Query Performance

The initial dataset consisted of more than 13 million rows (365 forecast days multiplied by approximately 13,000 grid points). Early implementations resulted in large storage requirements and slow query execution.

Solution:

Database optimization was performed by refining data types (e.g., replacing **DOUBLE PRECISION** with **REAL** and reducing integer sizes where possible) and redesigning indexing strategies. Redundant indexes were removed, while composite indexes on frequently queried fields were retained. These changes significantly reduced storage overhead and improved query performance.

Challenge 2: Efficient Spatial Search

Mapping a user's requested location to the nearest weather grid point using linear search resulted in unacceptable response times.

Solution:

A KD-tree spatial index was implemented in memory within the application layer. This reduced nearest-neighbor search complexity from $O(n)$ to $O(\log n)$, enabling fast responses without relying on specialized database extensions.

Challenge 3: Map Rendering Performance

Displaying all grid points simultaneously on an interactive map caused browser performance issues.

Solution:

Server-side sampling was introduced, limiting the number of points returned for visual overview while preserving full precision for specific location queries. This approach maintained usability without sacrificing data accuracy.

2.6 Results and Conclusions

The final system meets the primary objectives defined at the beginning of the project. Users can query any global location and receive long-term forecast data according to their subscription tier. Performance benchmarks show that most queries execute within sub-second response times, and the user interface remains responsive under realistic workloads.

Overall, WeatherSight demonstrates that large-scale weather datasets can be transformed into a practical and usable platform through careful architectural and algorithmic choices.

2.7 Lessons Learned

Several key lessons emerged during the project:

- **Early database design decisions** have a significant impact on performance and scalability.
- **Hybrid processing approaches**, such as combining database queries with in-memory algorithms, can outperform purely database-driven solutions.
- **Modular UI components** accelerate development and improve consistency across the system.

In retrospect, introducing full authentication earlier and allocating additional time for automated testing would have further improved the development process. These insights provide a strong foundation for future extensions of the system.

2.8 Project Metrics and Goal Validation

To evaluate whether WeatherSight achieved its goals, we defined a small set of measurable metrics focusing on functionality, performance, usability, and maintainability. These metrics were selected to reflect the main risks of the project: handling large-scale data efficiently while keeping the platform simple to use.

Functional completeness

Metric: The system supports end-to-end workflows for location-based search and forecast visualization, including subscription-based limitations.

Validation: Manual scenario testing was performed using predefined user flows (e.g., search by city → nearest grid match → forecast retrieval → visualization). The platform consistently returned valid results for a variety of geographic locations and supported both short-horizon and long-horizon forecasts according to user tier. Saved searches and dashboard features were verified through repeated runs.

Performance and responsiveness

Metric: Forecast queries should return results in sub-second time under typical usage, with stable UI responsiveness.

Validation: After database and indexing optimizations and the introduction of the KD-tree nearest-neighbor search, typical queries completed quickly and remained within an acceptable response window. The KD-tree reduced spatial lookup time and minimized unnecessary database load, while composite indexing supported efficient forecast retrieval. In addition, map performance remained stable due to server-side sampling rather than rendering all grid points simultaneously.

Usability and clarity

Metric: Users should be able to complete a basic search and interpret the results without requiring meteorological expertise.

Validation: The interface was iteratively refined using informal persona-based evaluation (e.g., agricultural planner, logistics manager). Charts, maps, and tables were adjusted to emphasize trends and reduce visual clutter. The result is a workflow that is consistent and intuitive: users search → explore results visually → save or compare results.

Maintainability

Metric: The project should remain extendable after completion, including the ability to ingest new datasets and evolve features.

Validation: The system was structured with separation between the UI, API routes, and database access logic. Core services such as spatial matching are encapsulated (e.g., within the WeatherGeoService), making future updates

manageable. Clear environment configuration and deployment compatibility (e.g., serverless hosting) further support maintainability.

Global applicability and provider independence

Metric: The platform should support global geographic coverage and remain independent of any single forecasting provider.

Validation: WeatherSight operates on a global geographic grid and accepts location-based queries for any coordinate worldwide. The internal data schema and ingestion process are designed to accommodate forecast datasets from different providers, as long as they conform to the required spatial and temporal structure. No provider-specific logic is embedded in the core system, confirming that the platform can be reused or extended by different forecasting organizations.

Conclusion: Based on the above validation, WeatherSight meets the primary project objectives defined during Phase A. Remaining limitations (such as replacing mock authentication with production-grade authentication and expanding automated tests) are documented and treated as future work rather than unresolved requirements.

2.9 Summary and Future Work

WeatherSight demonstrates how long-term weather data can be transformed from a large, complex dataset into a practical platform that supports planning and decision-making. The main contributions of the project include global grid-based search, interactive visualization, and a scalable architecture that balances database efficiency with application-layer computation (notably through KD-tree spatial indexing). In addition, the provider-agnostic and globally scalable design of WeatherSight positions it as a flexible platform that can be adopted by multiple forecasting organizations or used as an independent interface for long-term climate analytics.

While the current implementation successfully supports the intended workflows, several improvements were identified as realistic next steps:

- **Authentication and security hardening:** Replace mock authentication with full Supabase Auth, enforce JWT-based access control, and implement database Row-Level Security policies.
- **Automated testing:** Add unit and integration tests for core services (spatial matching, query building, subscription enforcement), and expand UI testing for the main workflows.

- **Data scaling enhancements:** If the dataset expands significantly, consider partitioning strategies and caching for frequently requested queries.
- **Alerts and notifications:** Extend the saved-search functionality with real alerts (email/push) triggered by threshold conditions or forecast changes.
- **Advanced analytics:** Add comparative tools across multiple locations and richer trend summaries to support industry use cases more directly.

These improvements are feasible within the existing architecture and would move the system closer to a production-grade platform.

Appendix A – User Guide (Operating Instructions)

A.1 System Overview

WeatherSight is a web-based platform designed for searching, analyzing, and visualizing long-term weather forecast data. The system is intended for end users who need to explore weather conditions across geographic locations and time horizons without requiring technical or meteorological expertise.

WeatherSight operates on a global geographic grid consisting of approximately 13,000 points. Each grid point contains daily forecast data for up to 365 days. When a user performs a search, the system automatically identifies the most relevant grid point and retrieves the associated forecast data. Results are presented using interactive charts, maps, and tables that support exploratory analysis and comparison.

This guide describes the **nominal (successful) operational flow** of the system and focuses on typical user interactions.

A.2 Getting Started

A.2.1 Accessing the System

Users can access WeatherSight either through a deployed web environment or by running the system locally during development.

link for deployed web environment: <https://weathersight.vercel.app/>

WeatherSight is designed for modern web browsers and supports both desktop and mobile displays.

A.2.2 Authentication

For the purposes of this project, WeatherSight uses a mock authentication mechanism. Users sign in to access personalized features such as saved searches, usage tracking, and subscription-based limitations.

Once authenticated, the user is redirected to the main dashboard, which serves as the central entry point for all system functionality.

A.2.3 Subscription Tiers

User access is governed by subscription tiers. Each tier defines limits on system usage and available forecast horizons:

- **Basic** – limited daily queries and short forecast horizon
- **Standard** – moderate daily queries and extended forecast horizon
- **Professional** – high query limits and long forecast horizon
- **Enterprise** – unlimited access and full forecast range

Subscription tiers control:

- Maximum number of queries per day
- Allowed forecast date range
- Number of saved searches

These constraints are enforced automatically by the system.

A.3 Core System Operations

A.3.1 Location-Based Weather Search

This is the primary and most common usage scenario.

Operational steps:

1. Enter a city name or geographic location (e.g., *Tel Aviv*) in the search field.
2. Select a specific date or date range using the calendar component.
3. Execute the search.
4. View the returned weather data, including temperature, precipitation, wind, and solar radiation.

Behind the scenes, the system converts the location into geographic coordinates, identifies the nearest grid point using an in-memory KD-tree, and retrieves the relevant forecast data.

A.3.2 Parameter-Based Search

This mode allows users to identify locations that meet predefined weather conditions.

Operational steps:

1. Navigate to the Parameter Search page.
2. Define desired weather criteria (e.g., temperature range, precipitation threshold).
3. Select the relevant date or date range.
4. Execute the search to receive a list of matching locations.

This workflow is useful for scenario exploration rather than location-specific analysis.

A.4 Managing User Data

A.4.1 Saved Searches

Users can save successful searches for future reuse.

Available actions:

- Save a search configuration
- Access saved searches from the dashboard
- Re-run saved searches using original parameters

A.4.2 Results Workspace

All executed searches are stored in the Results Workspace.

Users can:

- Review previous results
- Compare multiple searches
- Analyze trends across time horizons

A.5 Data Visualization

WeatherSight provides several visualization mechanisms to support analysis:

- Interactive charts for long-term trends
- Geographic maps with spatial context
- Data tables for detailed inspection and export

All visualizations are interactive and synchronized with user-selected parameters.

A.6 Summary

WeatherSight enables users to explore large-scale weather datasets through intuitive workflows and visual tools. By supporting both location-focused and parameter-driven analysis, the system allows users to identify patterns, compare scenarios, and manage results within a unified interface.

Appendix B – Maintenance Guide (Developer Instructions)

B.1 Purpose

This document is intended for developers and system maintainers responsible for the continued operation, maintenance, and extension of the WeatherSight platform after project completion.

It describes the system environment, installation steps, maintenance procedures, and scalability considerations required to ensure long-term usability.

B.2 System Environment Requirements

B.2.1 Software Infrastructure

Frontend

- Runtime: Node.js 18.x or higher
- Framework: Next.js 14 (App Router)
- Language: TypeScript
- Package management: npm or yarn

Backend and Database

- Database platform: Supabase (PostgreSQL)
- Core tables:
 - `weather_forecasts`
 - `weather_grid`
 - `users`
 - `saved_searches`

External Services

- Geocoding: OpenStreetMap Nominatim API
- Spatial indexing: In-memory KD-tree implementation

Production Considerations

- Supabase handles database availability and scaling
- CDN usage is recommended for static assets
- Monitoring tools should be integrated for observability

B.3 Installation and Initial Setup

B.3.1 Project Setup

1. Clone the project repository.
2. Navigate to the frontend directory.
3. Install dependencies using the selected package manager.
4. Create a `.env.local` file and configure:
 - Supabase project URL
 - Supabase anonymous key

B.3.2 Database Preparation

1. Execute database migration scripts in the defined order.
2. Populate the database using the provided data ingestion scripts.
3. Verify that grid points and forecast records were inserted correctly.

B.4 Running the System

To start the development environment, run the development server. Once started, the system is accessible through a web browser at the configured address.

B.5 System Maintenance

B.5.1 Data Management

- Add new weather datasets in CSV format
- Execute ingestion scripts to load new data
- Validate data completeness and consistency

Database maintenance tasks include:

- Monitoring query performance
- Maintaining indexes on core tables
- Performing regular backups
- Archiving obsolete data if required

B.5.2 Code Maintenance

Frontend

- Update dependencies periodically
- Apply security patches
- Deploy updates using platforms such as Vercel

API

- Maintain version control discipline
- Preserve backward compatibility

- Update documentation when APIs change

B.6 Monitoring and Troubleshooting

Performance monitoring should include:

- API response times
- Database query performance
- Frontend load and interaction times

Operational considerations include:

- Monitoring geocoding service rate limits
- Tracking subscription usage
- Ensuring data freshness

B.7 Scalability Considerations

Database

- Introduce read replicas for heavy loads
- Apply date-based partitioning if data volume increases
- Consider caching for frequently accessed queries

Application

- Utilize serverless execution for API routes
- Use CDN distribution for static content
- Apply load balancing where required

B.8 Security Considerations

B.8.1 Current Limitations

- Authentication is currently mock-based
- API access control is minimal

B.8.2 Production Recommendations

- Implement Supabase Authentication with JWT
- Enforce rate limiting
- Apply Row-Level Security policies
- Ensure sensitive configuration is excluded from version control